Refactorización del backend — IcfesApp

Rama	`refactor` (sale de `App_test`, commit `663dfaf`)
Última actualización	30-10-2026
Estado	Fases 1 y 2 terminadas · Fases 3 a 6 pendientes

Este documento registra **cada cambio** de la refactorización: qué se cambió, por qué y cómo se usa. Está pensado para que un desarrollador que llega al proyecto entienda la estructura sin tener que leer todo el código.

---

1. Mapa rápido del proyecto

La solución sigue una arquitectura por capas. Cada capa solo conoce a las de abajo:

```
IcfesApp.API            → Controladores HTTP. Heredan de ApiControllerBase.
   |
IcfesApp.Infrastructure → Implementación de los servicios (EF Core, Identity, JWT, correo).
   |
IcfesApp.Application    → Interfaces de servicios (IXxxService), DTOs, Result y mappers (ToDto).
   |
IcfesApp.Domain         → Entidades, enums, constantes y QuestionBuilder. No depende de nada.
```

Regla para agregar una funcionalidad nueva:

1. Entidad en `Domain/Entities` (si hace falta).
2. DTOs en `Application/<Modulo>/Dtos` y su mapper en `Application/<Modulo>/<Modulo>Mappings.cs`.
3. Interfaz `IXxxService` en `Application/Common/Interfaces`, que devuelve `Result` / `Result<T>`.
4. Implementación `XxxService` en `Infrastructure/Content` y su registro en `DependencyInjection.cs`.
5. Controlador en `API/Controllers` que **hereda de** `ApiControllerBase`.

---

2. Herencia e interfaces usadas

Tipo	Hereda de / implementa	Para qué
`ApiControllerBase` *(nuevo, abstracto)*	`ControllerBase`	Base de **todos** los controladores: `CurrentUserId` y traducción de `Result` → HTTP.
`AuthController`, `QuestionsController`, … (9)	`ApiControllerBase`	Ya no repiten código de usuario actual ni de manejo de errores.
`Result`	—	Resultado de una operación sin valor (éxito / errores / tipo de error).
`Result<T>`	`Result`	Igual, pero con un valor de retorno.
`AuthResult`	`Result` *(antes era una clase suelta)*	Agrega los datos de sesión (tokens, usuario, 2FA).
`IXxxService` → `XxxService`	interfaz → implementación	Los controladores dependen de la interfaz, no de la clase concreta.
`QuestionBuilder` *(nuevo)*	— (`sealed`)	Única forma de construir una `Question` válida.

---

3. Registro de cambios

Fase 1 — Errores y seguridad

1.1 Los estudiantes podían ver las respuestas correctas 🔒

- **Antes:** `GET /api/questions` solo pedía estar logueado y `QuestionDto` incluye `IsCorrect`.
- **Ahora:** todo `QuestionsController` exige la política `RequireTeacher` (Teacher o Admin). El estudiante sigue recibiendo sus preguntas por `/api/practice-sessions`, que nunca revela la respuesta antes de tiempo.
- **Archivo:** `API/Controllers/QuestionsController.cs`
- **Impacto en el front:** ninguno; solo el panel de administración usa ese endpoint.

1.2 Crear o editar una pregunta borraba las imágenes 🐞

- **Antes:** `ImageUrl`, `JustificationImageUrl` y la imagen de cada opción no se guardaban al crear/editar, y al editar se perdían las imágenes de las opciones. `QuestionDto` tampoco las devolvía.
- **Ahora:** las peticiones aceptan `imageUrl` (enunciado), `justificationImageUrl` y `options[].imageUrl`, y `QuestionDto` / `QuestionOptionDto` las devuelven.
- **Archivos:** `Application/Questions/Dtos/*`, `Infrastructure/Content/QuestionService.cs`
- **Impacto en el front:** compatible. Para no perder imágenes al editar desde el panel, el front debe reenviar las `imageUrl` que recibió.

1.3 El código HTTP del login dependía del texto del mensaje 🐞

- **Antes:** `AuthController` buscaba palabras como "*confirmar tu correo*" o "*bloqueada*" dentro del mensaje para decidir entre 401/403/423. Cambiar un texto rompía la API.
- **Ahora:** `Result` tiene `ErrorType` (`Validation`, `NotFound`, `Unauthorized`, `Forbidden`, `Locked`). El servicio indica el tipo y `ApiControllerBase.ToErrorResponse` decide el código.

ErrorType	HTTP
`Validation`	400
`Unauthorized`	401
`Forbidden`	403 (correo sin confirmar)
`NotFound`	404
`Locked`	423 (cuenta bloqueada)
- **Archivos:** `Application/Common/Models/ErrorType.cs`, `Result.cs`, `Infrastructure/Security/AuthService.cs`	

1.4 `register-staff` le entregaba al admin los tokens del usuario nuevo 🔒

- **Antes:** el admin creaba un profesor y recibía el access/refresh token **del profesor**.
- **Ahora:** devuelve solo los datos del usuario creado (`UserInfoDto`).
- **Impacto en el front:** ninguno; el front no leía esa respuesta.

1.5 `/api/auth/session` devolvía datos incompletos 🐞

- **Antes:** `firstName` y `lastName` vacíos y sin `schoolId` (se armaba solo con lo que trae el JWT).
- **Ahora:** lee el usuario de la base de datos (`IAuthService.GetCurrentUserAsync`) y devuelve todo.
- **Impacto en el front:** compatible (misma forma, ahora con datos).

1.6 Datos demo con contraseña conocida en todos los entornos 🔒

- **Antes:** `DataSeeder` (usuarios `studentX@test.com` / `Test12345!`) se ejecutaba siempre.
- **Ahora:** solo en `Development`. Roles y admin por defecto se siguen creando en todos los entornos.
- **Archivo:** `API/Program.cs`

1.7 Cualquiera podía ver planes inactivos 🔒

- **Antes:** `GET /api/plans?includeInactive=true` era público.
- **Ahora:** `includeInactive` solo tiene efecto para un Admin; para los demás se ignora.
- **Archivo:** `API/Controllers/PlansController.cs`

1.8 Sesión inexistente respondía 400 en vez de 404 🐞

- `POST /api/practice-sessions/{id}/answers` y `/finish` ahora responden **404** si la sesión no existe o es de otro estudiante.

1.9 Reportes usaba roles escritos a mano

- `[Authorize(Roles = "Teacher,Admin")]` → `[Authorize(Policy = Policies.RequireTeacher)]` (misma regla, definida en un solo lugar).

Fase 2 — Patrones y código limpio

2.1 Patrón Builder para preguntas (`QuestionBuilder`)

- **Problema:** una `Question` se armaba a mano en 3 lugares (crear, editar y seeder) y cada uno olvidaba algo (así nació el error 1.2). Las reglas solo se validaban en el servicio; el seeder no las validaba.
- **Solución:** `Domain/Builders/QuestionBuilder.cs`, la **única** forma de construir una pregunta. Valida las reglas del negocio:
- enunciado y justificación obligatorios,
- al menos 2 opciones,
- exactamente 1 opción correcta,
- todas las opciones con texto.

```
var questionBuilder = QuestionBuilder.ForSubject(subjectId)
    .WithStatement("¿Cuánto es 2 + 2?", statementImageUrl: null)
    .WithJustification("2 + 2 = 4")
    .WithDifficulty(QuestionDifficulty.Easy)
    .CreatedBy(teacherUserId)
    .AddOption("4", isCorrect: true)
    .AddOption("5", isCorrect: false);

var validationErrors = questionBuilder.Validate();   // lista vacía = válida
if (validationErrors.Count == 0)
{
    Question question = questionBuilder.Build();     // lanza excepción si no es válida
}
```

- **Quién lo usa:** `QuestionService.CreateAsync`, `QuestionService.UpdateAsync` (con `BuildOptionsFor`) y `DataSeeder`. En el seeder, una pregunta mal escrita en el JSON se omite en vez de romper el arranque.

2.2 Clase base de controladores (`ApiControllerBase`) — herencia

- **Problema:** `CurrentUserId` estaba copiado en 5 controladores (y en 2 más se calculaba en línea), y `ToActionResult` estaba copiado en 4.
- **Solución:** `API/Controllers/ApiControllerBase.cs` (abstracta):
- `CurrentUserId` → Id del usuario del JWT.
- `ToActionResult(Result)` → 204 si fue bien.
- `ToActionResult(Result<T>)` → 200 con el valor si fue bien.
- `ToErrorResponse(Result)` → código según `ErrorType`, con el cuerpo `{ "errors": [...] }`.
- Los 9 controladores heredan de ella y quedaron más cortos.

2.3 Un solo tipo `Result` — herencia

- **Antes:** había 4 tipos: `Result`, `Result<T>`, `OperationResult` y `AuthResult` (este último sin relación con los demás).
- **Ahora:** `OperationResult` se eliminó (se usa `Result`) y `AuthResult` **hereda** de `Result`. Todos tienen `Succeeded`, `Errors` y `ErrorType`.

2.4 Mappers `ToDto()` — un solo lugar por DTO

- **Problema:** `SchoolDto` se armaba 3 veces, `PlanDto` 2 veces, y la lógica de `IsUsable` (códigos) e `IsCurrentlyActive` (suscripciones) se recalculaba en los servicios aunque ya existe en la entidad.
- **Solución:** una clase estática de métodos de extensión por módulo:

Archivo	Métodos
`Application/Questions/QuestionMappings.cs`	`Question.ToDto()`, `QuestionOption.ToDto()`
`Application/Practice/PracticeSessionMappings.cs`	`ToDto()`, `ToResultDto()`, `ToAnswerResultDto()`
`Application/Schools/SchoolMappings.cs`	`School.ToDto()`
`Application/Subjects/SubjectMappings.cs`	`Subject.ToDto()`
`Application/Plans/PlanMappings.cs`	`Plan.ToDto()`
`Application/InvitationCodes/InvitationCodeMappings.cs`	`InvitationCode.ToDto()` (usa `IsUsable` de la entidad)
`Application/Subscriptions/SubscriptionMappings.cs`	`Subscription.ToDto()` (usa `IsCurrentlyActive` de la entidad)

2.5 Comportamiento en las entidades (dominio rico)

- `Question.CorrectOption`: devuelve la opción correcta (antes `Options.First(o => o.IsCorrect)` repetido en 4 sitios).
- `StudentAnswer.IsAnswered` y `StudentAnswer.RegisterSelectedOption(option)`: registrar una respuesta es ahora un solo método.
- Se documentaron con `<summary>` las propiedades de `Question` y `StudentAnswer`.

2.6 `AuthService` más legible

- `CompleteSignInAsync`: paso final común del login normal y del login con Google (pedir 2FA o emitir tokens). Antes estaba duplicado.
- `BuildUserInfo` y `DescribeErrors`: helpers que antes estaban repetidos.
- Constante `RecoveryCodeCount` en vez del número `10` suelto.

2.7 Rendimiento en sesiones de práctica

- **Iniciar sesión:** antes cargaba **todas** las preguntas de la materia con sus opciones para elegir unas pocas. Ahora trae solo los Ids, elige al azar y carga únicamente las elegidas

(`PickRandomQuestionsAsync`).

- **Buscar la sesión del estudiante:** el filtro por dueño ahora va en la consulta SQL, no después de cargarla.
- **Seeder de preguntas:** antes hacía una consulta por cada pregunta del JSON (~230). Ahora hace una sola.
- Consultas de solo lectura con `AsNoTracking()` .

2.8 Nombres claros de variables

Se eliminaron las variables de una letra y los nombres genéricos en los archivos tocados:

Antes	Ahora
`q` , `s` , `o` , `a` , `u` , `ic` , `ps` , `ur` , `rt` (lambdas)	`question` , `session` , `option` , `answer` , `user` , `invitationCode` , `practiceSession` , `userRole` , `refreshToken`
`result`	`createUserResult` , `resetPasswordResult` , `changePasswordResult` , `confirmEmailResult` …
`stored`	`storedRefreshToken`
`key`	`authenticatorKey`
`code` (era la entidad completa)	`invitationCode` · y `codeValue` para el texto
`item` , `opt`	`questionSeed` , `optionSeed`
`total` , `correct`	`totalQuestionCount` , `correctAnswerCount`
`payload`	`googlePayload`
`token` / `body` en correos	`passwordResetToken` , `emailConfirmationToken` , `emailHtmlBody`
`ex`	`exception`
`db` (Program.cs)	`dbContext`
`i` / `var q = questions[i]`	`questionIndex` / `var question = questions[questionIndex]`
`sanJoseSub` , `liceo`	`sanJoseSubscription` , `liceoModernoSchoolId`
Números sueltos (`6` , `4` , `-1` , `10`)	Constantes: `RandomSuffixLength` , `groupLength` , `UnlimitedUses` , `RecoveryCodeCount`

También se borraron comentarios de ruido como `// <--- AGREGADO AQUÍ (ESTE ERA EL ERROR)` y `// Método corregido` .

---

4. Cambios de comportamiento de la API (para el front)

Endpoint	Cambio	¿Rompe el front?
`GET/POST/PUT/DELETE /api/questions`	Solo Teacher/Admin (403 para estudiantes)	No (solo lo usa el panel admin)
`GET/POST/PUT /api/questions`	Nuevos campos `imageUrl`, `justificationImageUrl`, `options[].imageUrl`	No
`POST /api/auth/register-staff`	Devuelve `UserInfoDto` en vez de tokens	No (el front no lee la respuesta)
`GET /api/auth/session`	Ahora trae `firstName`, `lastName`, `schoolId`	No
`POST /api/practice-sessions/{id}/answers` y `/finish`	404 si la sesión no existe (antes 400)	No
`GET /api/plans?includeInactive=true`	Se ignora si no es Admin	No

Ningún endpoint cambió de nombre todavía (eso es la fase 5).

---

5. Pruebas realizadas

Compilación: **0 errores, 0 advertencias.** Prueba manual levantando la API compilada en un puerto aparte (`5299`) con una base de datos nueva:

#	Prueba	Resultado
1	Login con contraseña incorrecta	401 ✅
2	`/api/auth/session` de un estudiante	Trae nombre, apellido y `schoolId` ✅
3	Estudiante consulta `/api/questions`	403 ✅
4	Profesor consulta `/api/questions`	234 preguntas, 70 con imagen, campos de imagen presentes ✅
5	Crear pregunta sin opción correcta	400 con el mensaje del builder ✅
6	Crear pregunta con imágenes	Se guardan las del enunciado y la opción ✅
7	Editar pregunta (cambia correcta e imagen de opción)	204 y datos correctos al leerla ✅
8	Borrar pregunta	204 ✅
9	Iniciar sesión de práctica	3 preguntas, sin exponer `isCorrect` ✅
10	Responder	Devuelve si fue correcta, opción correcta y justificación ✅
11	Responder en sesión finalizada	400 "La sesión ya fue finalizada." ✅
12	Finalizar sesión inexistente	404 ✅
13	Admin crea profesor (`register-staff`)	200 con el usuario, sin tokens ✅
14	Planes inactivos como anónimo	Se ignora el filtro ✅
15	Reporte de profesor / suscripción actual	200 ✅

---

6. Pendiente

Fase	Contenido	Estado
3	Dividir `AuthService` (registro, login, tokens, contraseña, 2FA, correo, Google) · `ApplicationUserFactory` · Strategy para suscripción Particular/Institucional	Pendiente
4	**Proxy:** proxy de protección que exige suscripción activa para iniciar práctica · proxy de caché para materias y planes	Pendiente
5	Renombrar endpoints a nombres claros y eliminar los duplicados (requiere cambiar también el front)	Pendiente
6	Renombrar `School` → `Institution` (con migración) · dividir `DataSeeder` por entidad	Pendiente

Pendiente técnico: `StudentAnswer.AnsweredAtUtc` recibe la fecha actual al crear la sesión, aunque la pregunta aún no se haya respondido. Para hacerla `nullable` hace falta una migración, y la herramienta `dotnet-ef` no está instalada en este equipo (`dotnet tool install --global dotnet-ef`).